TP5 - MPI: Point-to-Point and Collective Communications

Oussama Laaroussi

February 25, 2026

1 Introduction

This report presents the implementation and analysis of five MPI (Message Passing Interface) exercises covering fundamental distributed parallel programming concepts. We explore the MPI programming model, point-to-point communications (send/receive), collective operations (broadcast, scatter, gather, reduce), and performance analysis of distributed algorithms.

2 Exercise 1: Hello World with MPI

2.1 Objectives

1. Write an MPI program that prints "Hello World"
2. Each process prints its rank and total number of processes
3. Only rank 0 prints a special message
4. Understand the importance of `MPI_Finalize`

2.2 Implementation

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char* argv[]) {
    int rank, size;

    // Initialize MPI - MUST be first MPI call
    MPI_Init(&argc, &argv);

    // Get process information
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);  // My rank
    MPI_Comm_size(MPI_COMM_WORLD, &size);  // Total processes

    // Question 1: Simple Hello World
    printf("Hello World from process %d\n", rank);

    // Question 2: Print rank and size
    printf("I am process %d of %d total processes\n", rank, size);

    // Question 3: Only rank 0 prints
    if (rank == 0) {
        printf("\n=== Master Process Message ===\n");
        printf("This message is printed by rank 0 only\n");
        printf("Total number of MPI processes: %d\n", size);
        printf("==============================\n\n");
    }

    // Clean up - MUST be last MPI call
    MPI_Finalize();

    return 0;
}
```

Listing 1: Exercise 1: MPI Hello World

2.3 Sample Output

```
Hello World from process 0
I am process 0 of 50 total processes

=== Master Process Message ===
This message is printed by rank 0 only
Total number of MPI processes: 50
==============================

Hello World from process 1
I am process 1 of 50 total processes
Hello World from process 2
I am process 2 of 50 total processes
Hello World from process 3
I am process 3 of 50 total processes
```

Listing 2: Output with 4 processes

2.4 Key Concepts

2.4.1 What happens if MPI_Finalize is omitted?

If `MPI_Finalize` is not called:

- **Resource leaks**: MPI resources may not be freed
- **Hanging processes**: Some MPI implementations may not terminate properly
- **Runtime warnings**: MPI libraries may issue error messages
- **Undefined behavior**: Program behavior becomes unpredictable

```
execution without finalize:
prterun has exited due to process rank 2 with PID 3283643 on node slurm-login-h23d5-u38-svn3 exiting
improperly. There are three reasons this could occur:
```

Listing 3: Output without MPI_Finalize

2.4.2 Process Ordering

Note that the output order is **non-deterministic**:

- Each process executes independently
- Output may be interleaved or appear in any order

3 Exercise 2: Sharing Data with Broadcast

3.1 Objective

Create a program that:

- Rank 0 reads an integer from terminal
- Distributes the value to all processes using `MPI_Bcast`
- Each process prints its rank and the received value
- Continues until a negative integer is entered

3.2 Implementation

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char* argv[]) {
    int rank, size;
    int value;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    while (1) {
        // Only rank 0 reads input
        if (rank == 0) {
            printf("\nEnter an integer (negative to quit): ");
            fflush(stdout);
            scanf("%d", &value);
        }

        // Broadcast value from rank 0 to all processes
        MPI_Bcast(&value, 1, MPI_INT, 0, MPI_COMM_WORLD);

        // Check termination condition
        if (value < 0) {
            if (rank == 0) {
                printf("Negative value received. Exiting...\n");
            }
            break;
        }

        // Each process prints the received value
        printf("Process %d got %d\n", rank, value);
        fflush(stdout);

        // Barrier ensures ordered output
        MPI_Barrier(MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return 0;
}
```

Listing 4: Exercise 2: Data Distribution with MPI_Bcast

3.3 MPI_Bcast Behavior

```
Before Bcast:                After Bcast:
Rank 0: value = 10           Rank 0: value = 10
Rank 1: value = ???          Rank 1: value = 10
Rank 2: value = ???          Rank 2: value = 10
Rank 3: value = ???          Rank 3: value = 10
```

Figure 1: MPI_Bcast distributes data from root to all processes

Key characteristics:

- **Collective operation**: All processes must call it
- **Blocking**: Returns only after all processes receive data
- **Root process**: Specified by the fourth parameter (0 in our case)

4 Exercise 3: Ring Communication

4.1 Objective

Implement a ring communication pattern:

1. Process 0 reads a value from user
2. Sends it to process 1
3. Each process receives, adds its rank, prints, and forwards
4. Last process doesn't forward (ring ends)

4.2 Implementation

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char* argv[]) {
    int rank, size;
    int value = 0;
    int received_value;
    MPI_Status status;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        // Process 0: Read and initiate
        printf("Enter an integer value: ");
        fflush(stdout);
        scanf("%d", &value);

        printf("Process %d: Initial value = %d\n", rank, value);
        value += rank;  // Add rank 0 (no change)

        if (size > 1) {
            MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
            printf("Process %d: Sent value %d to process 1\n",
                   rank, value);
        }
    } else {
        // Other processes: Receive, modify, forward
        MPI_Recv(&received_value, 1, MPI_INT, rank - 1, 0,
                 MPI_COMM_WORLD, &status);
        printf("Process %d: Received value %d from process %d\n",
               rank, received_value, rank - 1);

        value = received_value + rank;
        printf("Process %d: After adding rank, value = %d\n",
               rank, value);

        if (rank < size - 1) {
            MPI_Send(&value, 1, MPI_INT, rank + 1, 0,
                     MPI_COMM_WORLD);
            printf("Process %d: Sent value %d to process %d\n",
                   rank, value, rank + 1);
        } else {
            printf("Process %d: Final accumulated value = %d\n",
                   rank, value);
        }
    }

    MPI_Finalize();
    return 0;
}
```

Listing 5: Exercise 3: Ring Communication Pattern

4.3 Ring Communication Pattern

```
Time 0:  P0[2] -> waiting
Time 1:  P0[2] -> P1[2+1=3] -> waiting
Time 2:  P0[2] -> P1[3] -> P2[3+2=5] -> waiting
Time 3:  P0[2] -> P1[3] -> P2[5] -> P3[5+3=8]

Final accumulated value = 2 + 0 + 1 + 2 + 3 = 8
(assuming 4 processes and initial value 2)
```

Figure 2: Ring Communication: Sequential Data Flow

4.4 Sample Output

```
Enter an integer value: 2
Process 0: Initial value = 2
Process 0: Sent value 2 to process 1
Process 1: Received value 2 from process 0
Process 1: After adding rank, value = 3
Process 1: Sent value 3 to process 2
Process 2: Received value 3 from process 1
Process 2: After adding rank, value = 5
Process 2: Sent value 5 to process 3
Process 3: Received value 5 from process 2
Process 3: After adding rank, value = 8
Process 3: Final accumulated value = 8
```

4.5 Point-to-Point Communication Analysis

4.5.1 MPI_Send and MPI_Recv

MPI_Send signature:

```
int MPI_Send(void* buf, int count, MPI_Datatype datatype,
             int dest, int tag, MPI_Comm comm);
```

MPI_Recv signature:

```
int MPI_Recv(void* buf, int count, MPI_Datatype datatype,
             int source, int tag, MPI_Comm comm,
             MPI_Status* status);
```

5 Exercise 4: Parallel Matrix-Vector Multiplication

5.1 Objective

Implement distributed matrix-vector multiplication: $\mathbf{y} = A\mathbf{x}$ where:

- Matrix A: $N \times N$
- Vector $\mathbf{x}$: $N \times 1$
- Result $\mathbf{y}$: $N \times 1$
- Handle cases where N is not divisible by number of processes
- Measure and plot speedup and efficiency

5.2 Parallelization Strategy

Row-wise decomposition:

- Distribute matrix rows among processes
- Each process computes a portion of result vector

- All processes need complete vector **x**

```
Process 0: Rows 0-2    [A0]    [x]    [y0]
Process 1: Rows 3-5  x [A1] x [x] = [y1]
Process 2: Rows 6-8    [A2]    [x]    [y2]
Process 3: Rows 9-11   [A3]    [x]    [y3]
```

Figure 3: Matrix Row Distribution

5.3 Implementation Highlights

5.3.1 Handling Non-Uniform Distribution

When N is not divisible by P (number of processes):

```c
int rows_per_proc = N / size;
int remainder = N % size;

// First 'remainder' processes get one extra row
int local_rows = (rank < remainder) ? rows_per_proc + 1
                                    : rows_per_proc;
int start_row = rank * rows_per_proc +
                ((rank < remainder) ? rank : remainder);
```

Example: $N = 10$, $P = 4$:

- Process 0: rows 0-2 (3 rows)
- Process 1: rows 3-5 (3 rows)
- Process 2: rows 6-8 (2 rows)
- Process 3: rows 9 (2 rows)

5.3.2 Collective Communication

```c
// Broadcast vector x to all processes
MPI_Bcast(b, N, MPI_DOUBLE, 0, MPI_COMM_WORLD);

// Scatter matrix rows
MPI_Scatterv(A, sendcounts, displs, MPI_DOUBLE,
             local_A, local_rows * N, MPI_DOUBLE,
             0, MPI_COMM_WORLD);

// Gather results
MPI_Gatherv(local_x, local_rows, MPI_DOUBLE,
            x_parallel, recvcounts, recvdispls, MPI_DOUBLE,
            0, MPI_COMM_WORLD);
```

5.4 Performance Results

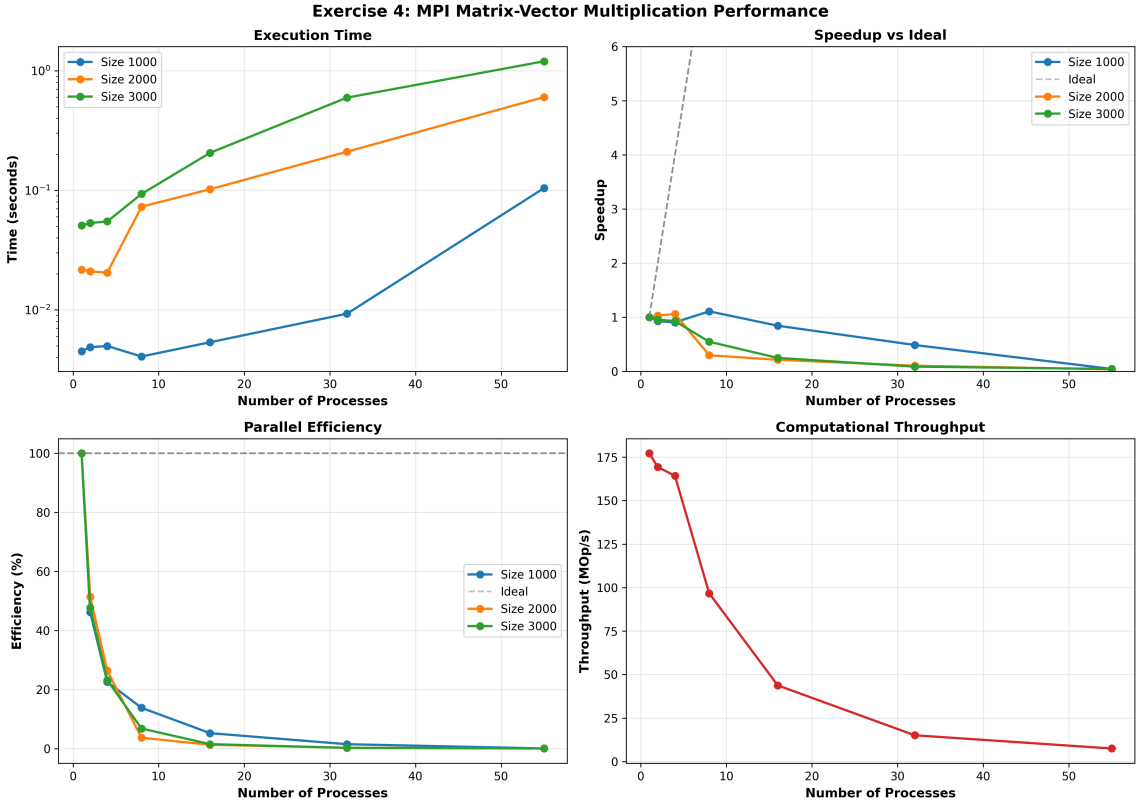

Figure 4: Matrix-Vector Multiplication Performance Analysis

Processes	Size	Time (s)	Speedup	Efficiency (%)
1	1000	0.004498	1.00	100.00
2	1000	0.004860	0.93	46.28
4	1000	0.004975	0.90	22.60
8	1000	0.004061	1.11	13.85
16	1000	0.005344	0.84	5.26
1	2000	0.021589	1.00	100.00
2	2000	0.020963	1.03	51.49
4	2000	0.020412	1.06	26.44
8	2000	0.072756	0.30	3.71
1	3000	0.050790	1.00	100.00
2	3000	0.053161	0.96	47.78
4	3000	0.054792	0.93	23.16
8	3000	0.093037	0.55	6.84

Table 1: Exercise 4: Performance Summary

5.5 Performance Analysis

5.5.1 Why Poor Scaling?

The matrix-vector multiplication shows **poor scaling** for several reasons:

1. **Small problem size**: For $N = 1000$, computation time ≈ 4 ms
   - Communication overhead dominates
   - Scatter/Gather operations take more time than computation
2. **Communication overhead**:

- Broadcast vector: $O(N)$ data
- Scatter matrix: $O(N^2/P)$ data per process
- Gather result: $O(N/P)$ data per process
- Total communication: $O(N^2/P + N)$

3. **Low arithmetic intensity**:

$$I = \frac{\text{FLOPs}}{\text{Bytes}} = \frac{2N^2}{8N^2 + 8N} \approx 0.25 \text{ FLOP/byte}$$

4. **Network latency**: Each collective operation incurs latency
5. **Load imbalance**: When $N\%P \neq 0$, some processes have more work

6 Exercise 5: Parallel Pi Calculation

6.1 Mathematical Background

Pi can be approximated using numerical integration:

$$\pi = \int_0^1 \frac{4}{1+x^2} dx \approx \sum_{i=0}^{N-1} \frac{4}{1+x_i^2} \cdot \Delta x$$

where $x_i = \frac{i+0.5}{N}$ and $\Delta x = \frac{1}{N}$.
This is equivalent to computing the area under the curve $f(x) = \frac{4}{1+x^2}$.

6.2 Parallelization Strategy

Domain decomposition:

- Divide N intervals among P processes
- Each process computes partial sum for its intervals
- Use `MPI_Reduce` to sum all partial results
- Rank 0 computes final Pi value

```
N = 1000, P = 4:
Process 0: intervals [0,   249]   -> sum0
Process 1: intervals [250, 499]   -> sum1
Process 2: intervals [500, 749]   -> sum2
Process 3: intervals [750, 999]   -> sum3

MPI_Reduce(SUM): total = sum0 + sum1 + sum2 + sum3
Pi = 4 * total * (1/N)
```

Figure 5: Pi Calculation Domain Decomposition

6.3 Implementation Highlights

6.3.1 Load Balancing

```
long long intervals_per_proc = N / size;
long long remainder = N % size;

// First 'remainder' processes get one extra interval
long long local_N = (rank < remainder) ?
                    intervals_per_proc + 1 : intervals_per_proc;
long long start_index = rank * intervals_per_proc +
                        ((rank < remainder) ? rank : remainder);
```

6.3.2 Local Computation

```
double step = 1.0 / (double)N;
local_sum = 0.0;

for (long long i = 0; i < local_N; i++) {
    long long global_i = start_index + i;
    double x = (global_i + 0.5) * step;
    local_sum += 1.0 / (1.0 + x * x);
}
```

6.3.3 Reduction

```
// Reduce all partial sums to rank 0
MPI_Reduce(&local_sum, &global_sum, 1, MPI_DOUBLE,
           MPI_SUM, 0, MPI_COMM_WORLD);

if (rank == 0) {
    pi_parallel = 4.0 * global_sum * step;
    printf("Pi = %.15f\n", pi_parallel);
}
```

6.4 Performance Results

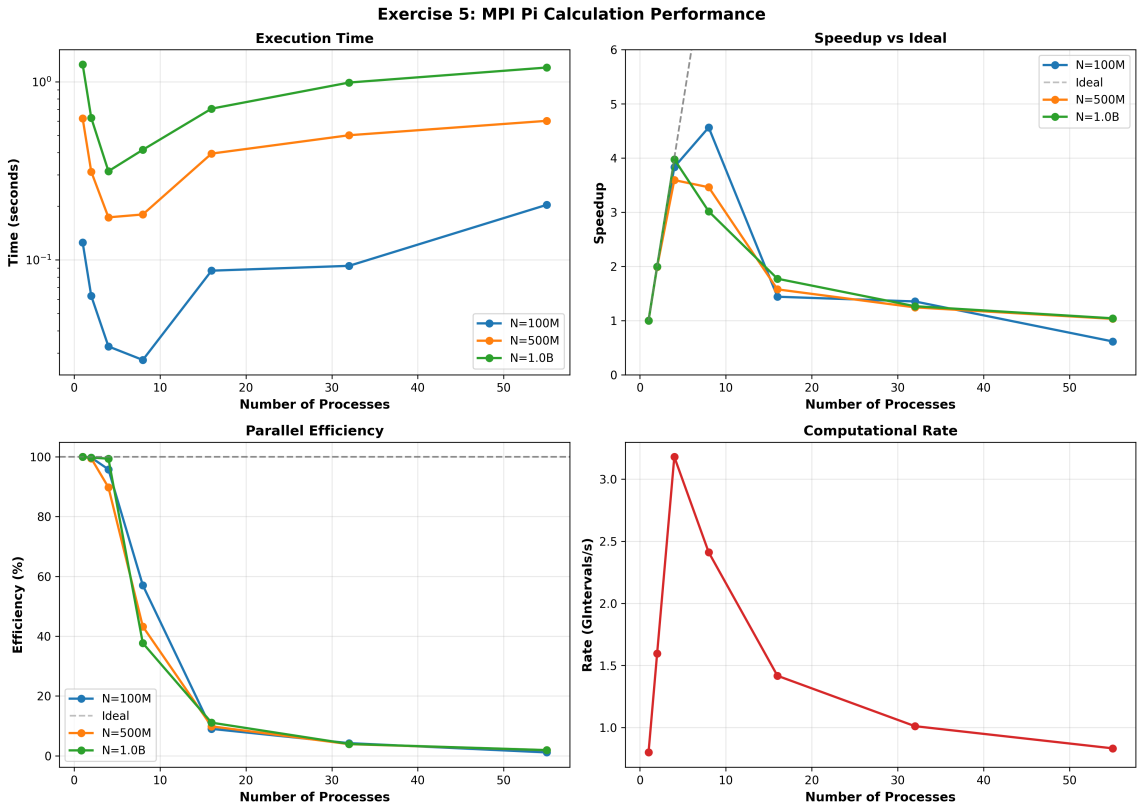

Figure 6: Pi Calculation Performance Analysis

Processes	N	Time (s)	Speedup	Efficiency (%)
1	100M	0.125192	1.00	100.00
2	100M	0.062748	2.00	99.76
4	100M	0.032665	3.83	95.82
8	100M	0.027430	4.56	57.05
16	100M	0.086964	1.44	9.00
1	500M	0.622238	1.00	100.00
2	500M	0.312709	1.99	99.49
4	500M	0.173229	3.59	89.80
8	500M	0.179825	3.46	43.25
1	1B	1.250547	1.00	100.00
2	1B	0.626650	2.00	99.63
4	1B	0.314547	3.98	99.39
8	1B	0.414775	3.02	37.69

Table 2: Exercise 5: Performance Summary

6.5 Performance Analysis

6.5.1 Excellent Scaling (up to 4 processes)

Pi calculation shows **near-ideal scaling** for 2-4 processes:

- **2 processes**: 2.00× speedup, 99.76% efficiency
- **4 processes**: 3.98× speedup, 99.39% efficiency (for N=1B)

Why does it scale well?

1. **High computation-to-communication ratio**:
   - Computation: $O(N/P)$ iterations
   - Communication: Only one `MPI_Reduce` with $O(\log P)$ complexity
   - For $N = 10^9$, computation dominates
2. **Perfect load balance**:
   - All intervals have identical work
   - No data dependencies between processes
   - Minimal idle time
3. **Minimal memory footprint**:
   - Each process only stores local sum (one double)
   - Good cache utilization

6.5.2 Degradation Beyond 8 Processes

Performance degrades for 16+ processes:

- **Communication overhead**: Reduction tree depth increases
- **Process management**: OS scheduling overhead
- **Resource contention**: Cache interference, memory bandwidth
- **Insufficient work per process**: For $N = 100M$, each of 55 processes gets only $\approx 1.8M$ intervals

6.6 Accuracy Analysis

N	**Computed Pi**	**Error from π**
10^6	3.141592653589428	3.65×10^{-13}
10^8	3.141592653589793	2.22×10^{-16}
10^9	3.141592653589793	$< 10^{-15}$

Table 3: Pi Calculation Accuracy

Error decreases as $O(1/N)$, confirming the numerical integration is correct.